Fractal 프로토콜 이론적 배경

개요

Fractal은 홀로그래픽 IOP(Holographic Interactive Oracle Proof)를 기반으로 한 zkSNARK 프로토콜로, 전처리(preprocessing) 기능을 지원하여 검증자의 시간 복잡도를 O(log N)으로 달성합니다. 이는 기존의 Aurora나 Ligero 프로토콜과 달리 인덱서(indexer)를 통한 사전 계산을 활용하여 효율성을 크게 개선한 것입니다.

홀로그래픽 IOP의 개념

홀로그래픽 증명 시스템의 정의

홀로그래픽 증명 시스템은 다음과 같은 특징을 가집니다:

1. **전처리 단계**: 회로나 제약 시스템에 대한 사전 계산을 수행
2. **인덱서 역할**: 전처리된 정보를 구조화하여 저장
3. **효율적 검증**: 전처리된 정보를 활용하여 빠른 검증 수행

홀로그래픽 IOP에서 증명자는 다음과 같은 구조를 가집니다:

```
홀로그래픽 IOP = (인덱서, 증명자, 검증자)
```

여기서:

- **인덱서(Indexer)**: 제약 시스템 M에 대해 인덱스 I = Indexer(M)을 생성
- **증명자(Prover)**: 인덱스 I와 증인 w를 사용하여 증명 π = Prover(I, w) 생성
- **검증자(Verifier)**: 인덱스 I와 공개 입력 x를 사용하여 Verifier(I, x, π) 검증

전처리 기능의 장점

전처리 기능은 다음과 같은 핵심 장점들을 제공합니다:

1. 검증 시간 단축

- 전처리 없는 시스템: 검증자가 O(N) 시간 필요
- 홀로그래픽 시스템: 검증자가 O(log N) 시간으로 단축
- 이는 인덱서가 제약 시스템의 구조를 미리 분석하고 최적화하기 때문

2. 회로 재사용성

- 동일한 회로에 대해 인덱스를 한 번만 생성
- 여러 증명에서 동일한 인덱스 재사용 가능
- 대규모 시스템에서 전체적인 효율성 크게 향상

3. 메모리 효율성

- 검증자가 저장해야 하는 정보량 감소
- 인덱스는 원본 회로보다 훨씬 작은 크기
- 분산 시스템에서 검증자 노드의 부담 경감

인덱서(Indexer)의 역할

인덱서의 핵심 기능

Fractal의 인덱서는 `matrix_indexer` 클래스로 구현되며, 다음과 같은 핵심 기능을 수행합니다:

1. 행렬 분해와 인코딩

```
// R1CS 제약 시스템의 행렬 A, B, C를 분해
```

```
matrix_indexer<FieldT> A_indexer, B_indexer, C_indexer;



// 각 행렬을 row, col, val 다항식으로 변환

polynomial<FieldT> row_poly, col_poly, val_poly;
```

2. 도메인 설정과 최적화

- **인덱스 도메인 K**: 행렬의 비영 원소 개수보다 큰 크기
- **행렬 도메인 H**: 제약 조건과 변수를 식별하는 도메인
- **코드워드 도메인 L**: Reed-Solomon 인코딩을 위한 확장 도메인

3. 오라클 생성과 등록

```
oracle_handle row_oracle_handle_;

oracle_handle col_oracle_handle_;

oracle_handle val_oracle_handle_;

oracle_handle row_times_col_oracle_handle_;
```

O(log N) 검증자 시간 달성 원리

인덱서를 통한 O(log N) 검증 시간 달성은 다음과 같은 메커니즘으로 이루어집니다:

1. 사전 계산된 구조 활용

- 인덱서가 제약 시스템의 구조를 미리 분석
- 검증에 필요한 핵심 정보만 추출하여 저장
- 검증 시 전체 제약 시스템을 다시 처리할 필요 없음

2. 효율적인 쿼리 구조

- 인덱스를 통해 특정 위치의 값들을 빠르게 접근
- 머클 트리 구조를 활용한 O(log N) 인증 경로
- 무작위 쿼리의 수가 보안 매개변수에만 의존

3. 홀로그래픽 감소(Holographic Reduction)

```
검증 복잡도: O(N) → O(log N)
```

이는 다음과 같은 단계로 이루어집니다:

1. 전체 제약 시스템을 작은 부분 문제들로 분해
2. 각 부분 문제의 해답을 사전 계산
3. 검증 시 부분 문제들의 일관성만 확인

재귀적 구성과 확장성

재귀적 증명의 개념

Fractal은 재귀적 증명 구성을 지원하여 확장성을 크게 개선합니다:

1. 증명 합성(Proof Composition)

```
증명 π₁ + 증명 π₂ → 합성된 증명 π
```

- 여러 개의 작은 증명들을 하나의 큰 증명으로 결합
- 각 증명의 검증 과정 자체를 회로로 표현
- 재귀적으로 증명의 증명을 생성

2. 점진적 검증 가능 계산(IVC)

- 긴 계산 과정을 여러 단계로 분할
- 각 단계의 정확성을 개별적으로 증명
- 최종 결과의 정확성을 효율적으로 검증

확장성 개선 방법

1. 병렬 처리 지원

```
// 여러 인덱서를 병렬로 실행
std::vector<matrix_indexer<FieldT>> matrix_indexers_;
```

- 대규모 제약 시스템을 여러 부분으로 분할
- 각 부분을 독립적으로 인덱싱
- 병렬 처리를 통한 전체 처리 시간 단축

2. 메모리 효율적 구현

- 스트리밍 방식의 인덱스 생성
- 필요한 부분만 메모리에 로드
- 대용량 회로 처리 가능

3. 적응적 매개변수 조정

```
// 회로 크기에 따른 동적 매개변수 설정
fractal_iop_parameters<FieldT> params(
security_parameter,
pow_bits,
```

```
RS_extra_dimensions,

make_zk,

constraint_system

);
```

실용적 확장성 분석

1. 시간 복잡도 비교

| 단계 | 기존 시스템 | Fractal |

|------|----------|---------|

| 전처리 | - | O(N log N) |

| 증명 생성 | O(N log N) | O(N log N) |

| 검증 | O(N) | O(log N) |

2. 공간 복잡도 개선

- 인덱스 크기: O(N) → O(√N log N)
- 증명 크기: 기존과 유사한 $O(\log^2 N)$
- 검증자 메모리: O(N) → O(log N)

3. 통신 복잡도 최적화

- 인덱스 공유를 통한 통신 오버헤드 감소
- 배치 검증을 통한 네트워크 효율성 향상
- 분산 환경에서의 확장성 개선

다른 프로토콜과의 비교

Fractal vs Aurora

| 특성 | Aurora | Fractal |

|------|--------|---------|

| 전처리 | 없음 | 있음 |

| 검증 시간 | $O(\log^2 N)$ | $O(\log N)$ |

| 인덱스 크기 | - | $O(\sqrt{N} \log N)$ |

| 재사용성 | 낮음 | 높음 |

Fractal vs Ligero

| 특성 | Ligero | Fractal |

|------|--------|---------|

| 증명 크기 | $O(\sqrt{N})$ | $O(\log^2 N)$ |

| 검증 시간 | $O(\sqrt{N})$ | $O(\log N)$ |

| 전처리 | 없음 | 있음 |

| 투명성 | 완전 | 완전 |

보안성 분석

홀로그래픽 시스템의 보안 고려사항

1. 인덱스 무결성

- 인덱스 생성 과정의 정확성 보장
- 악의적인 인덱스 생성 방지
- 인덱스 검증 메커니즘 필요

2. 영지식성 보존

```
bool make_zk = true; // 영지식 모드 활성화
```

- 전처리 과정에서 비밀 정보 누출 방지
- 인덱스를 통한 증인 정보 추론 불가능
- 추가적인 마스킹 기법 적용

3. 사운드니스 유지

- 인덱서의 정확성이 전체 시스템 보안에 영향
- 홀로그래픽 감소 과정의 사운드니스 증명
- 재귀적 구성에서의 보안성 누적

이러한 이론적 배경을 바탕으로 Fractal 프로토콜은 전처리 기능을 통해 기존 시스템의 한계를 극복하고, 실용적인 대규모 zkSNARK 시스템 구축을 가능하게 합니다.